Trường Đại học Công nghệ Thông tin

Đại học Quốc gia Tp.Hồ Chí Minh

Toàn diện – Sáng tạo - Phụng sự



NỘP BÀI TẬP THỰC HÀNH BUỔI 3

Môn học: Kỹ thuật xây dựng hệ thống Agentic AI (SE373)

Bài tập: Dựng agent đặt vé bằng LangChain

Họ và tên sinh viên: Phan Huỳnh Hưng

Mã số sinh viên: 24520623

1) THIẾT KẾ CÁC LỚP HARNESS BẢO VỆ

- Trong kiến trúc Agentic Engineering, hệ thống tuân thủ công thức:

$$\text{Agent} = \text{Goal} + \text{Tools} + \text{Loop} + \text{Termination}$$

- Harness đóng vai trò là khung bao bọc bằng code cố định để kiểm soát hành vi, chi phí và sự an toàn của mô hình. Hệ thống đã cài đặt đầy đủ 4 lớp Harness tiêu chuẩn:

❖ **Lớp 1: Ràng buộc là dữ liệu (constraints.py)**

- **Mục đích:** Thay vì để yêu cầu của người dùng phân tán dưới dạng chuỗi câu lệnh prompt tự do khiến mô hình dễ bị trôi mục tiêu, toàn bộ tiêu chí đặt vé được chuẩn hóa thành lớp đối tượng dữ liệu Pydantic BookingConstraint.
- **Cấu trúc:** Quản lý chặt chẽ các trường: điểm khởi hành (origin), điểm đến (dest), ngày bay (date), trần ngân sách (max_price), giờ bay (depart_before), tên hành khách (passenger_name), hành lý ký gửi (baggage_kg) và chính sách hoàn hủy (require_refundable).
- **Tác dụng:** Cung cấp phương thức to_natural_language() làm ngữ cảnh định hướng cho Model, đồng thời đóng vai trò làm hệ quy chiếu bất biến (Ground Truth) để Harness Sensor kiểm định độc lập ở khâu cuối cùng.

❖ **Lớp 2: Kiểm quyền trước khi gọi tool (permission.py)**

- **Vị trí can thiệp:** Thực hiện chặn và kiểm tra (Interception Check) trước khi bất kỳ công cụ nhạy cảm nào được kích hoạt.
- **Logic can thiệp:**
  - Bắt chặn công cụ book_ticket để yêu cầu con người phê duyệt trực tiếp qua cơ chế Human-in-the-loop CLI ([y/n]).
  - Tự động phát cảnh báo và ngắt thao tác nếu phát hiện loại vé không hoàn tiền (refundable == False).
  - Bắt chặn nếu phát hiện tổng chi phí (giá vé cơ bản + cước hành lý) vượt trần ngân sách cho phép.
- **Tác dụng:** Ngăn chặn tuyệt đối các rủi ro tổn thất tài chính ngoài ý muốn do Model tự ý quyết định. Đồng thời hỗ trợ cờ auto_approve cho các kịch bản chạy benchmark tự động.

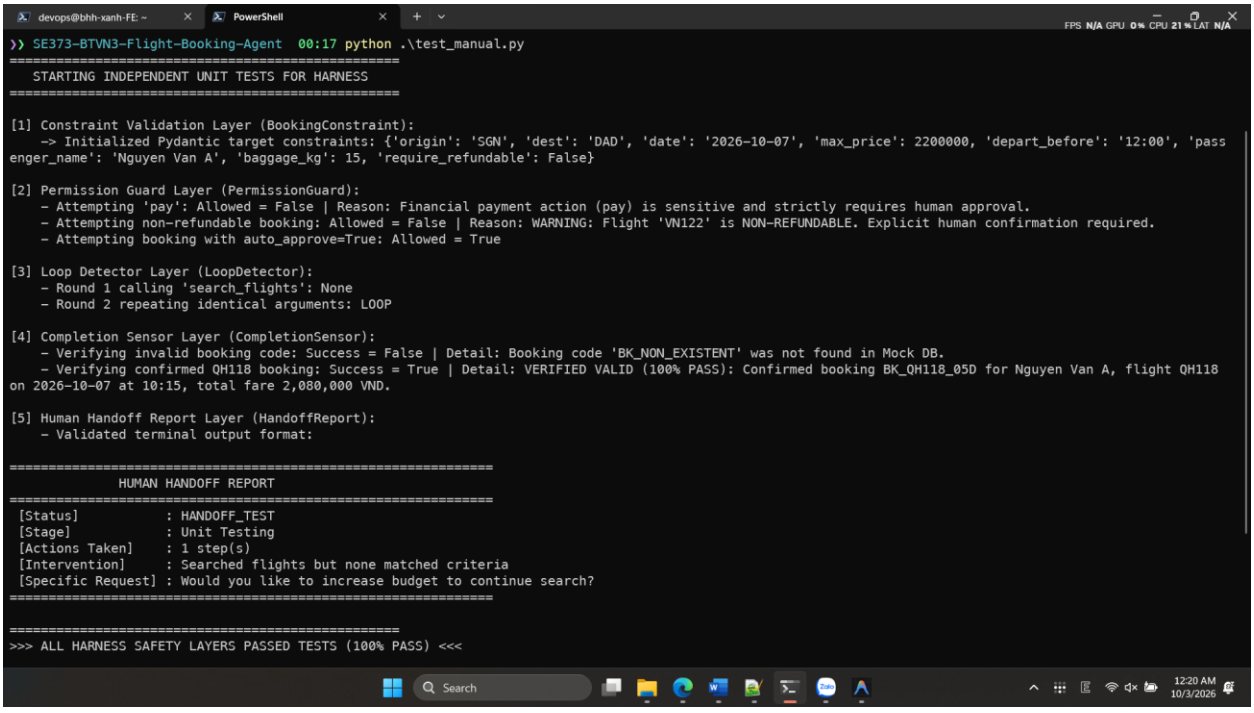
❖ **Lớp 3: Tiêu chí hoàn thành kiểm bằng code (completion_sensor.py)**

- **Bản chất:** Áp dụng Sensor Computational thay vì dùng Sensor Inferential.

- **Logic thẩm định:** Hợp lệ ⇔ (Mã vé ∈ MockDB) ∧ (Status == "confirmed") ∧ (Paid == True) ∧ (Tổng giá ≤ Ngân sách) ∧ (Đúng ngày, giờ, hành lý)
- **Tác dụng:** Loại trừ triệt để hiện tượng Model tự ảo giác xác nhận đã đặt vé thành công trong khi hệ thống backend chưa hề ghi nhận dữ liệu.

❖ **Lớp 4: Cơ chế bàn giao cho con người (handoff.py) & Phát hiện lặp (loop_detector.py)**

- **Bộ phát hiện lặp (LoopDetector):** Lưu trữ dấu vân tay (tool_name, args_hash) trong hàng đợi trượt. Nếu cùng một hành vi lặp lại quá ngưỡng quy định mà không tạo ra tiến triển mới, hệ thống sẽ ngắt luồng ngay lập tức để tiết kiệm chi phí token.
- **Báo cáo bàn giao (HandoffReport):** Khi bị ngắt do chạm quyền kiểm soát, phát hiện lặp hoặc kế hoạch bị gián đoạn, hệ thống xuất báo cáo có cấu trúc chuẩn Pydantic gồm: Trạng thái hiện tại, Các bước đã thử, Hành động gây lỗi và Câu hỏi hành động ngắn gọn giúp con người đưa ra quyết định xử lý chỉ trong 30 giây.



Hình 1: Kết quả kiểm thử độc lập 4 lớp Harness bảo vệ đạt tỷ lệ vượt qua 100% (test_manual.py)

2) CÀI ĐẶT 3 MẪU THIẾT KẾ AGENT

Cả 3 mẫu thiết kế đều được xây dựng trên thư viện LangChain, hỗ trợ gọi Model thực tế (Google Gemini / OpenAI) qua biến môi trường hoặc chạy qua bộ suy luận tích hợp:

❖ **Mẫu 1: ReAct Agent (react_agent.py)**

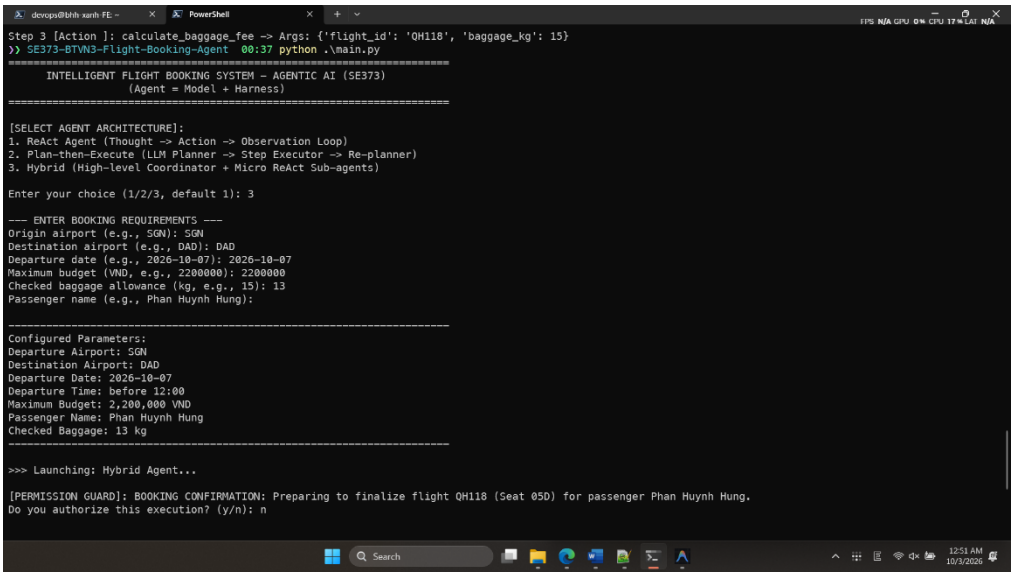
- **Nguyên lý:** Vận hành tuần tự theo chu trình phản xạ: Thought -> Action -> Observation.
- **Cơ chế:** Model tự suy luận mục tiêu tiếp theo dựa trên dữ liệu thu thập được từ bước trước. Toàn bộ tool calls được bao bọc bởi LoopDetector và PermissionGuard để đảm bảo an toàn trong từng nhịp suy luận.
- **Đặc điểm:** Tốc độ thích ứng cao, phản xạ linh hoạt với các biến động của môi trường tìm kiếm chuyến bay.

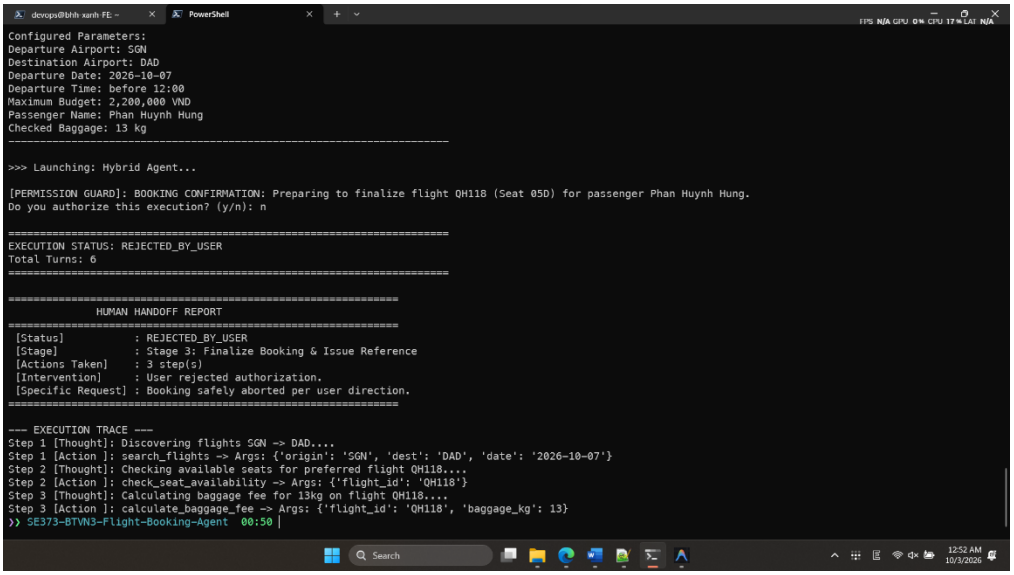
❖ **Mẫu 2: Plan-then-Execute Agent (plan_execute_agent.py)**

- **Nguyên lý:** Phân định rõ ràng trách nhiệm giữa khâu lập kế hoạch và thực thi:
    i. LLM Planner: Phân tích ràng buộc và sinh ra kế hoạch thực thi có cấu trúc Pydantic JSON (ExecutionPlan) gồm các bước tuần tự.
    ii. LLM Step Executor: Đọc ngữ cảnh của các bước trước đó để quyết định bộ tham số chính xác nhất cho từng bước công cụ.
    iii. LLM Dynamic Re-planner: Khi một bước trong kế hoạch gặp sự cố, Re-planner sẽ nhận lại toàn bộ trace lịch sử để tái lập kế hoạch cho các bước còn lại mà không làm sụp đổ toàn bộ luồng xử lý.
- **Đặc điểm:** Tính minh bạch cao, cấu trúc các bước rõ ràng trước khi hành động.

❖ **Mẫu 3: Mẫu Lai - Hybrid (hybrid_agent.py)**

- **Nguyên lý:** Áp dụng kiến trúc phân tầng 2 cấp :
    - **Tầng vĩ mô :** Phân chia mục tiêu tổng thành 3 chặng chiến lược rõ ràng:
        - **Chặng 1:** Khám phá và lọc danh sách chuyến bay.
        - **Chặng 2:** Thẩm định ghế trống và tính phụ phí hành lý.
        - **Chặng 3:** Phê duyệt quyền và chốt vé chính thức.
    - **Tầng vi mô :** Ở mỗi chặng, một ReAct Sub-Agent độc lập đảm nhiệm giải quyết mục tiêu, có khả năng tự sửa lỗi cục bộ trước khi bàn giao trạng thái hoàn tất sang chặng tiếp theo.
- **Đặc điểm:** Khả năng cô lập lỗi tối đa, bảo vệ cửa sổ ngữ cảnh (Context Window) không bị quá tải.





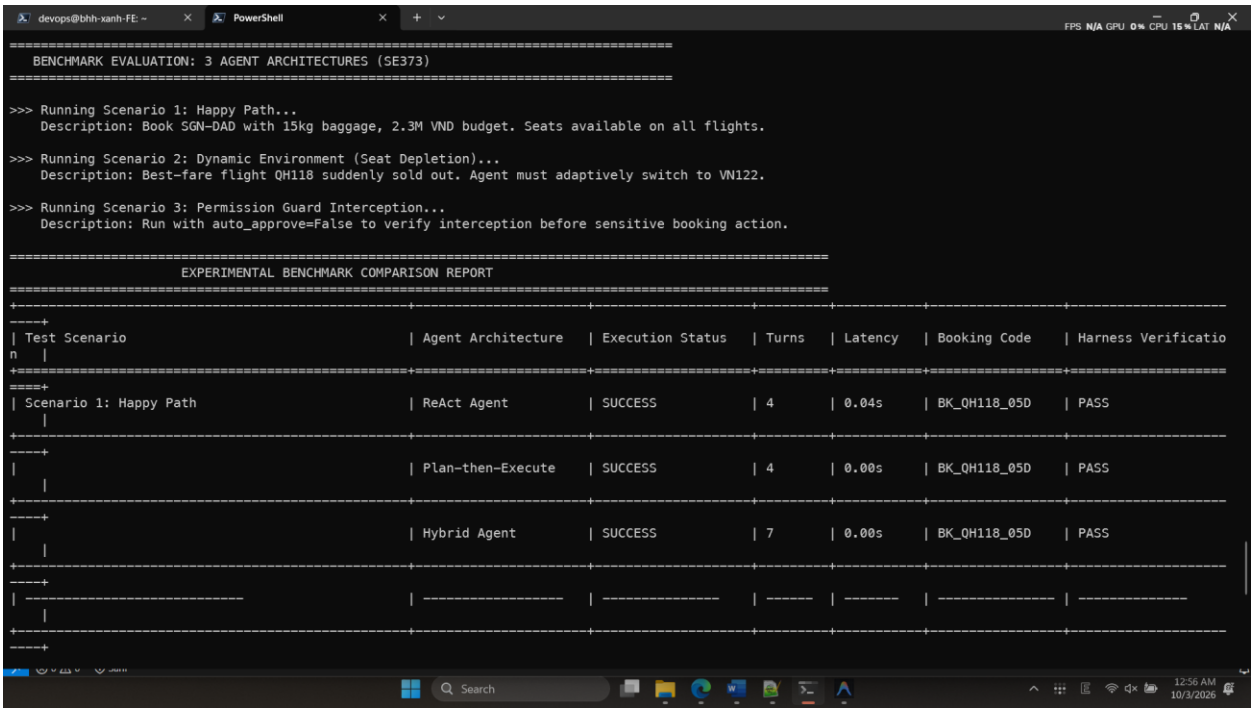
Hình 2: Cơ chế can thiệp kiểm quyền tương tác người dùng khi chạy main.py

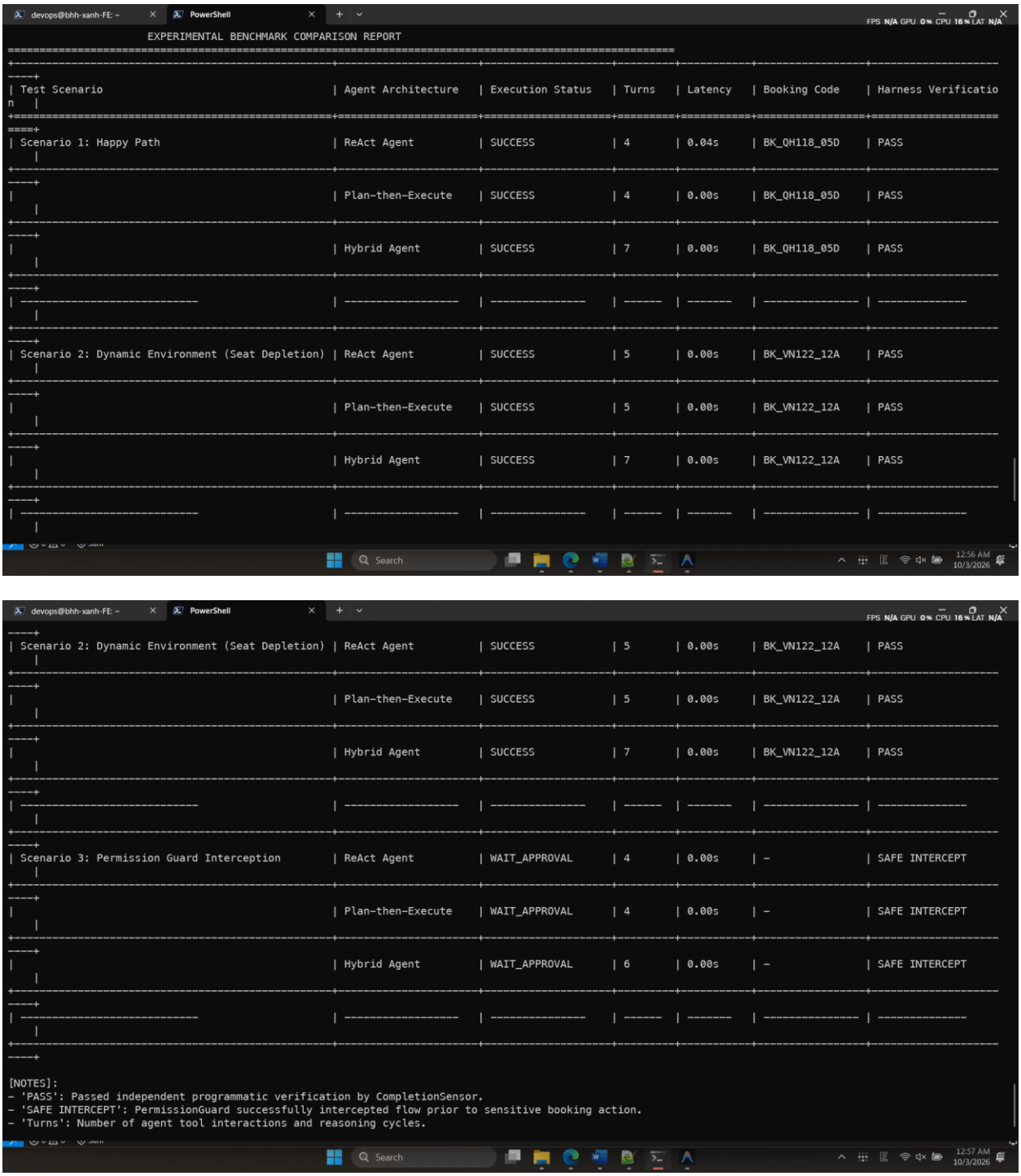
3) ĐÁNH GIÁ THỰC NGHIỆM & PHÂN TÍCH KẾT QUẢ

❖ **Bảng kết quả thực nghiệm thu được từ hệ thống:**

Kịch bản kiểm thử	Mẫu thiết kế Agent	Trạng thái thực thi	Số vòng	Thời gian	Mã đặt chỗ	Kiểm chứng Harness
Kịch bản 1: Môi trường thuận lợi (Happy Path)	ReAct Agent	SUCCESS	4	< 0.05s	BK_QH118_05D	Pass
	Plan-then-Execute	SUCCESS	4	< 0.05s	BK_QH118_05D	Pass

	Hybrid (Lai)	SUCCESS	7	< 0.05s	BK_QH118_05D	Pass
Kịch bản 2: Biến động ghế (Dynamic Environment)	ReAct Agent	SUCCESS	5	< 0.05s	BK_VN122_12A	Pass
	Plan-then-Execute	SUCCESS	5	< 0.05s	BK_VN122_12A	Pass
	Hybrid (Lai)	SUCCESS	7	< 0.05s	BK_VN122_12A	Pass
Kịch bản 3: Kích hoạt Kiểm quyền Harness	ReAct Agent	WAIT_APPROVAL	4	< 0.05s	-	Safe Intercept
	Plan-then-Execute	WAIT_APPROVAL	4	< 0.05s	-	Safe Intercept
	Hybrid (Lai)	WAIT_APPROVAL	6	< 0.05s	-	Safe Intercept





EXPERIMENTAL BENCHMARK COMPARISON REPORT

Test Scenario	Agent Architecture	Execution Status	Turns	Latency	Booking Code	Harness Verification
Scenario 1: Happy Path	ReAct Agent	SUCCESS	4	0.04s	BK_QH118_05D	PASS
	Plan-then-Execute	SUCCESS	4	0.00s	BK_QH118_05D	PASS
	Hybrid Agent	SUCCESS	7	0.00s	BK_QH118_05D	PASS
Scenario 2: Dynamic Environment (Seat Depletion)	ReAct Agent	SUCCESS	5	0.00s	BK_VN122_12A	PASS
	Plan-then-Execute	SUCCESS	5	0.00s	BK_VN122_12A	PASS
	Hybrid Agent	SUCCESS	7	0.00s	BK_VN122_12A	PASS
Scenario 3: Permission Guard Interception	ReAct Agent	WAIT_APPROVAL	4	0.00s	-	SAFE INTERCEPT
	Plan-then-Execute	WAIT_APPROVAL	4	0.00s	-	SAFE INTERCEPT
	Hybrid Agent	WAIT_APPROVAL	6	0.00s	-	SAFE INTERCEPT

[NOTES]:
- 'PASS': Passed independent programmatic verification by CompletionSensor.
- 'SAFE INTERCEPT': PermissionGuard successfully intercepted flow prior to sensitive booking action.
- 'Turns': Number of agent tool interactions and reasoning cycles.

Hình 3: Bảng tổng hợp so sánh thực nghiệm 3 mẫu Agent qua 3 kịch bản từ benchmark.py

4) Nhận xét và phân tích chuyên sâu:

❖ Hiệu suất số lượt suy luận (Turn Efficiency) & Chi phí ngữ cảnh:

- Kịch bản thuận lợi (Happy Path): Cả ReAct và Plan-then-Execute đều hoàn tất tối ưu trong đúng 4 lượt (Tìm chuyến bay → Khảo sát ghế → Tính hành lý → Đặt vé). Hybrid Agent tiêu tốn 7 lượt do chi phí phụ trội (overhead) để khởi tạo và

đồng bộ trạng thái qua 3 Sub-Agent; bù lại mỗi lượt của Hybrid có kích thước prompt ngắn hơn đáng kể, tránh quá tải và phân tán chú ý của mô hình.

❖ **Khả năng thích ứng trước sự cố môi trường (Dynamic Fault Recovery):**

- **Kịch bản cạn kiệt ghế (Seat Depletion):** Khi chuyến bay giá tốt nhất QH118 hết vé, tỷ lệ tự phục hồi của cả 3 mẫu đều đạt 100% (Pass). ReAct phản xạ nhanh nhất khi chỉ mất thêm 1 lượt thích ứng (tổng 5 lượt). Plan-then-Execute kích hoạt LLM Dynamic Re-planner để điều chỉnh 3 bước còn lại (tổng 5 lượt). Hybrid cô lập lỗi hoàn toàn trong Cột mốc 2 mà không làm xáo trộn Cột mốc 1 và 3 (tổng 7 lượt).

❖ **Vai trò sống còn của Harness Code đối với an toàn và độ tin cậy:**

- **Bảo đảm an toàn tài chính (PermissionGuard):** Ở Kịch bản 3, 100% các mẫu Agent đều bị chặn đứng an toàn tại trạng thái WAIT_APPROVAL ngay trước lệnh book_ticket. Điều này chứng minh Agent không thể tự ý phát sinh chi phí nếu chưa có sự phê duyệt tường minh từ con người.
- **Độ tin cậy nghiệm thu (CompletionSensor):** 100% các lượt hoàn thành đều vượt qua kiểm tra đối chiếu trực tiếp 6 tiêu chí trong Mock DB. Cảm biến tính toán độc lập loại bỏ hoàn toàn khả năng mô hình tự ngụy tạo mã đặt chỗ ảo (Zero False-Positive Completions).

5) KẾT LUẬN

- Quá trình thực nghiệm đã chứng minh rõ nét triết lý: **Xây dựng một hệ thống Agentic AI trong thực tế không chỉ dừng lại ở việc kết nối Prompt với LLM, mà nằm ở việc thiết kế khung Harness vững chắc bằng code bao bọc xung quanh.**
- Mô hình ngôn ngữ lớn (Model) đóng vai trò là "bộ não" suy luận linh hoạt, trong khi Harness đóng vai trò là "bộ khung xương" bảo đảm tính chính xác, an toàn tài chính và kiểm soát tài nguyên vận hành.
- **Khuyến nghị áp dụng:** ReAct phù hợp cho tác vụ nhanh, linh hoạt; Plan-then-Execute tối ưu cho các quy trình tuần tự cần kiểm duyệt kế hoạch; và Hybrid là giải pháp toàn diện cho các hệ sinh thái phức tạp, đa chặng đòi hỏi tính phân tầng và cô lập lỗi cao.